

COMS 4130 Final Project Design Document

By: Taylor Bauer, Abby Van Der Brink, & Trevor List

1. Background of Infrapy

Infrapy is a Python program developed by Los Alamos National Laboratory (LANL) for the detection of infrasonic sound waves in nuclear treaty monitoring. It uses the Adaptive Fisher Detector algorithm (AFD), which compares signals across recordings from 4 ground sensors and marks a detection when signal strength exceeds a threshold that adjusts based on background noise. The program processes sensor data through a pipeline: raw sensor recordings are loaded and converted into arrays, scanned in short time chunks to identify signal direction and strength, and then the Fisher detector decides if a real detection occurred. The full pipeline we focus on is depicted below. This project focuses on `beamforming_new.py`, specifically targeting 4 core functions that are in the pipeline:

- **run_fd**: where the detection decision happens (Adaptive Fisher Algorithm here); compares signal strength against the adaptive threshold across consecutive time windows and flags a detection if the signal is strong and consistent enough
- **calc_det_thresh**: computes the adaptive threshold that `run_fd` uses based on current background noise levels, it makes the detector more or less sensitive depending on conditions
- **run_fk**: this is the signal processing entry point; it loads the 4 ground sensor recordings and processes them in time windows for computing direction and strength of a signal
- **find_peaks**: this finds the strongest signal direction from the beamforming output of each window and passes it to the detector

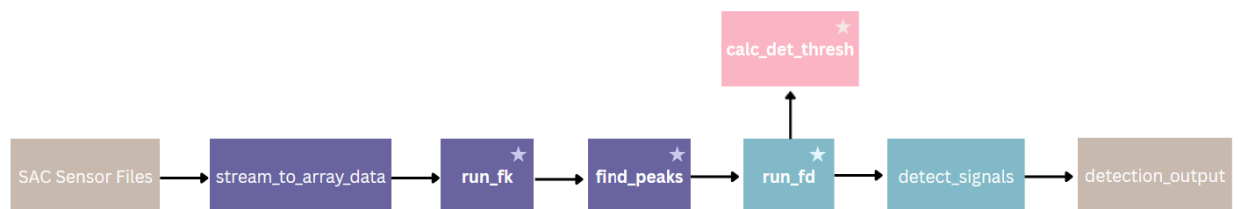


Figure 1: Diagram of Pipeline Focus

2. Set up

2.1 Setting up program (pyrite and cloning and such)

The infrapy repository was cloned and set up on the pyrite (ISU's Linux Server). The following dependencies were installed using pip: obspy, scipy, numpy, pyproj, pathos, pisces, ipython, pycallgraph2, and coverage. For test values, infrapy was run on infrasound sensor array recordings (YJ.BRP1-4.SAC) using test_beamforming.py as the test case for coverage and analysis.

Note: A bug was found in LANL's test_detection.py. detect_signals() was missing a required argument det_p_val which caused a crash at runtime, and was fixed by adding det_p_val=det_thresh to the function call.

2.2 Call Graph

A call graph of the detection pipeline was generated using pycallgraph2, tracing all function call relationships across the core detection functions. With this, a Python script was created to extract function call relationships from the source code into call.json for the database.

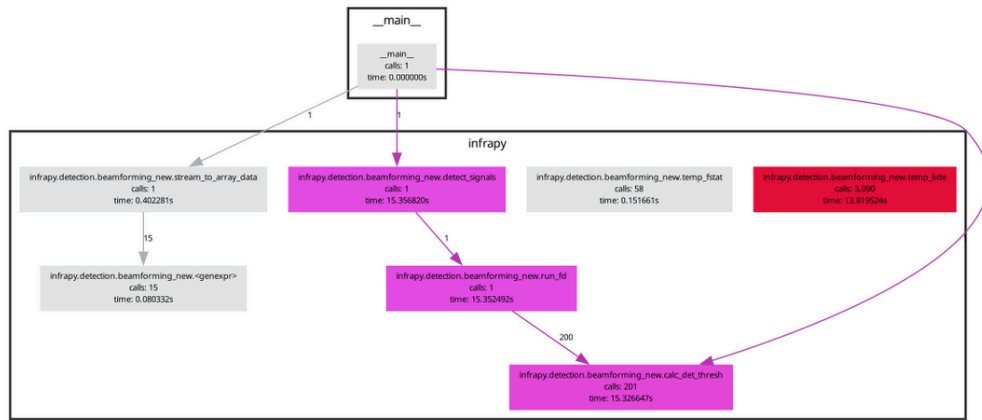


Figure 2.2: Call Graph of Detection Pipeline

2.3 Coverage

coverage.py was used and run on test_beamforming.py, results were output to coverage.json for the database.

2.4 Data Flow

A Python script was used with the ast module to track the detection variables across the 4 target functions as well as detect_signals and beam_window. These variables were fstat_vals, det_mask, thresh, beam_peaks, back_az_vals, trc_vel_vals, dets, det_p_val, fstat_ref_peak, back_az_95conf, and thresh_vals. The script looked for DEFINE, USE, and CALL events and the results were output to dataflow.json for the database.

2.5 Fuzzing

Fuzzing was performed using a hypothesis-based approach, with manually crafted edge cases rather than relying solely on automated input generation. This was done to test dataset size, array shape alignment, numeric values in data, and parameter sensitivity in `run_fk`, `find_peaks`, `run_fd`, and `calc_det_thresh` within `beamforming_new`. The fuzzing scope was applied to individual functions and to the full pipeline.

2.6 Populating SQLite Database

An SQLite Database was set up with four tables: `static_analysis`, `metadata`, `coverage`, and `fuzzing`. The database was then populated with JSON data generated in the earlier steps. Each JSON file was run through a Python program that matched the data with the attribute rows on the database tables and inserted it into the tables.



Figure 2.6.1: SQLite Schema

3. Queries & Motivation

3.1 Call Graph Reachability Queries

To better understand the program and its structure, seven queries were created to help answer developer questions, such as call paths between two functions, what function is called directly by another function, and what are the callees of a certain function. Initially, the query was set up in a Python file that used basic query functions. The call graph was traversed through BFS/DFS search to discover callers and callees.

3.1.1 Queries Designed:

1. `is_caller <function_A> <function_B>`: Is A a direct or indirect caller of B?
Returns True/False + path
2. `shortest_path <function_A> <function_B>`: What functions can A reach (directly or transitively)?
3. `callers <function_A>`: What functions eventually call B?
4. `callees <function_B>`: What is the shortest call chain from A to B?
5. `direct <function_A>`: What does A call directly (one hop)?
6. `who_calls <function_B>`: Direct callers of B
7. `list`: list all function in graph

3.1.2 Functions supported:

1. `run_fk`: signal processing entry point, processes sensor recordings in the time windows
2. `find_peaks`: finds the strongest signal direction from each time window and sends to the detector
3. `run_fd`: (AFD) where the actual detection decision happens! Decides if a signal is strong and consistent enough to count as a detection
4. `calc_det_thresh`: threshold computed for detection events based on background noise level

3.2 Coverage Lookup Queries

Developers are usually interested in knowing how much of their program has been tested. Four coverage-based queries were created to help answer questions such as the coverage of a specific Infrapy file, the lines that were never executed within a file, and what variables are data-dependent.

3.2.1 Queries Designed:

1. `uncovered <file>`: List every uncovered line in a file
2. `branch_coverage <file>`: Coverage % and uncovered branches for a file
3. `summary`: Overall coverage summary across all files
4. `Compare <file>`: Compare coverage across test runs

3.2.2 Known Files:

1. `test_beamforming.py`: Tutorial for ObsPy stream and trace manipulations
2. `beamforming_new.py`: Methods for reading in time series data, analyzing it, and identifying infrasonic signals using various beamforming methods.

3.3 Dataflow Dependency Queries

It is important that developers understand how variables flow through the program. To answer any questions a developer may have about the Infrapy dataflow, two queries were created to help developers better understand the program.

3.3.1 Queries Designed:

1. dataflow <var>: Where is a variable used/defined?
2. dependent <var_A> <var_B>: Are two variables data-dependent?

3.3.2 Known Variables:

1. thresh: detection threshold from the masked f-stat values
2. det_mask: values from beamforming results
3. fstat_vals: list of f-stat values from beamforming results
4. fstat_ref_peak: detection threshold
5. beam_peaks: array of beam peaks
6. thresh_vals: list of thresh values
7. det_p_val: Threshold p-value for declaring a detection
8. dets: List of identified detections including detection time, relative start, and end times of the detection, back azimuth, trace velocity, and f-stat.
9. back_az_vals: Back azimuth values defining polar slowness grid
10. back_az_95conf: length and width of back azimuth values
11. trc_vel_vals: Trace velocity values defining polar slowness grid
12. dets: List of identified detections including detection time, relative start and end times of the detection, back azimuth, trace velocity, and f-stat.

3.4 Fuzzing Queries

Fuzzing is a testing technique that helps developers identify security and stability flaws by providing invalid, random, or malformed data to trigger crashes, bugs, or vulnerabilities. The information given through pyfuzz tests was used to create six queries that would help a developer answer any questions they may have on the fuzz testing results.

3.4.1 Queries Designed:

1. summary: Overall pass/fail counts across all stages
2. failures: All expected_failure runs with error messages
3. stage <name>: All runs for a specific stage
4. inputs <name>: Find runs by input name (e.g. nan_inf_input)
5. channel <n>: All runs with a specific channel count (e.g. n = 2)
6. risky: Runs that passed but used out-of-range inputs

3.4.2 Known Fuzz Stages:

1. run_fk: signal processing entry point, processes sensor recordings in the time windows
2. find_peaks: finds the strongest signal direction from each time window and sends to the detector

3. `run_fd`: (AFD) where the actual detection decision happens! Decides if a signal is strong and consistent enough to count as a detection
4. `calc_det_thresh`: threshold computed for detection events based on background noise level

3.5 Natural Language Router

With the original query systems that were built, the user was required to enter the queries in a certain pattern (e.g. `dependent <var_a> <var_b>`). This is fine if the developer is aware of the set patterns, but it is not very convenient. To make querying easier for developers, a natural keyword router was developed for the InfraPy static analysis query system. It dispatches to one of the query systems depending on detected keywords. The keyword router searches for keywords such as ‘caller of,’ ‘dependent,’ or ‘coverage of’ and matches them to queries.

3.5.1 Example Natural Language Questions comparison:

Previous Patterned Query	Natural Language Variant
<code>is_caller <file_a> <file_b></code>	Is <code>detect_signals</code> a caller of <code>calc_det_thresh</code> ?
<code>branch_coverage <file></code>	What is the coverage of <code>beamforming_new.py</code> ?
<code>dependent <var_a> <var_b></code>	Are <code>thresh</code> and <code>det_mask</code> data-dependent?
<code>failures</code>	Show fuzzing failures

4. Internal workflow

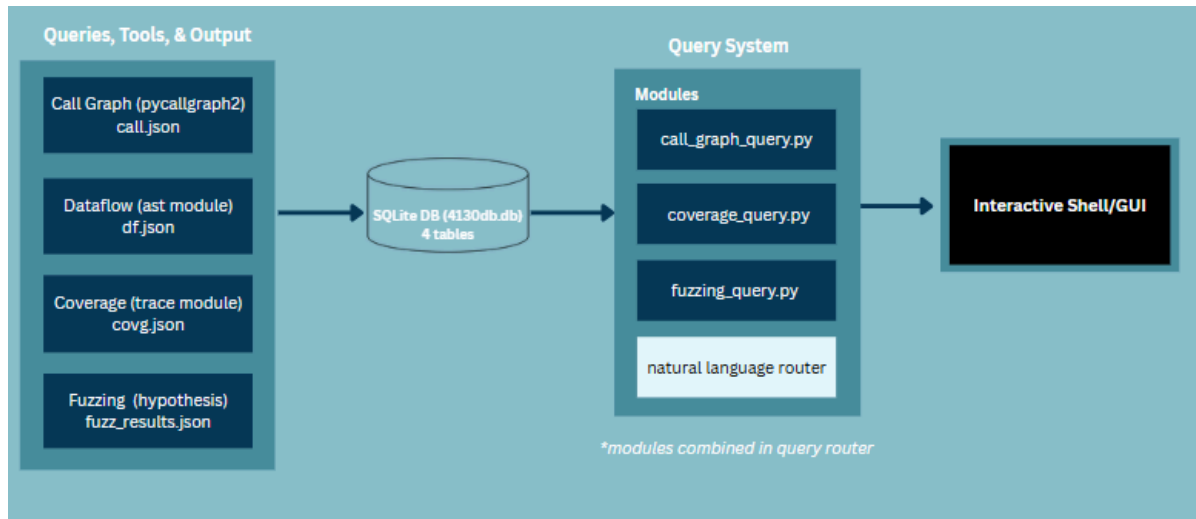


Figure 4.1.1: Internal Workflow Graph (can totally change it, but I just thought id add :))

5. Additional Features

5.1 Query Router UI

As an additional feature, an interactive user interface was created. A user-friendly interface makes a query system much more usable. The Query UI is set up in a single HTML file and uses Javascript for query routing and CSS for reactive components. A user can type a natural language question in the search bar and then click the run button to receive an answer to their query. The user can also click on the example queries on the right hand side of the search bar and the system will automatically answer the example query. The user can clear past queries with the clear button at the top of the UI. They can also scroll through all their past queries if they have done multiple all at once. The UI also shows the user what database is used by the router and whether the router is actually connected to the database.



Figure 5.1.1: Screenshot of Query Router UI running on Live Server

5.1.1 UI Suggested Queries:

Call Graph:

1. Is detect_signals a caller of calc_det_thresh?
2. What does run_fk directly call?
3. Who calls compute_beam_power?
4. What is the path from beam_window to project_Ab?
5. What can beam_window reach?
6. All callers of calc_det_thresh

Coverage:

1. What is the coverage of beamforming_new.py?

2. What lines are uncovered in beamforming_new.py?
3. Show overall coverage summary
4. Compare test runs for beamforming_new.py
5. What is the coverage of test_beamforming.py?

Dataflow:

1. Where is thresh used?
2. Are thresh and det_mask data-dependent?
3. Trace the det_p_val variable
4. Where is dets defined?
5. Are fstat_vals and det_mask data-dependent?

Fuzzing:

1. Show fuzzing summary
2. Show all fuzzing failures
3. Show fuzzing runs for stage run_fk
4. Show fuzzing runs for stage run_fd
5. Show risky fuzzing runs
6. Fuzzing channel 2

5. References